

AI闪购业务 - Agent开发经验记录

使用规则：每次新开发前必须阅读本文，避免重复踩坑。每次开发结束后更新本文，沉淀新经验。

一、架构决策记录（ADR）

ADR-001：Multi-Agent 编排框架选 LangGraph，不自己写状态机

决策时间：2026-05-07

背景：闪购 workflow 涉及选品→定价→文案→审核→投放 5个Agent节点，其中审核需要人机交互卡点（暂停→人工审核→继续），workflow 状态需要持久化。

选了 LangGraph 的原因：

- `StateGraph` 原生支持条件边 + 循环（审核驳回退回选品），自己写状态机要处理大量边界情况
- `interrupt` + `Command(resume=...)` 原生人机交互，不需要自己实现审批队列、状态恢复
- `checkpoint`（如 `SqliteSaver`）自动持久化 workflow 状态，服务重启不丢进度
- `stream_mode="updates"` 支持逐步输出每个 Agent 的推理结果，前端体验好
- Agent 节点内可直接 bind 工具函数（调用推荐模型、数据服务），不需要自己写 function calling 胶水代码

已知坑：

- LangGraph 的 `interrupt` 机制在高并发时需要确保 checkpoint 的线程安全（MVP 阶段并发低，暂不担心）
- `Command(resume=...)` 的 API 在 2025 年初有 breaking change，注意版本锁定
- `StateGraph` 的嵌套子图（subgraph）在复杂场景下调试困难，MVP 阶段保持扁平结构

ADR-002：MVP 阶段 MVP 使用模拟数据，不全量对接真实数据源

决策时间：2026-05-07

背景：真实闪购业务涉及 5+ 数据源（交易、商品、用户、竞品、商圈），对接周期至少额外 2 周，且涉及数据权限。

选了模拟数据的原因：

- 6周交付 MVP 的时间约束下，优先验证核心 Agent 逻辑和用户体验
- 特征工程抽象层设计好接口（`FeatureExtractor` 基类），后续切换到真实数据源只需实现新子类
- 模拟数据按真实业务逻辑生成（商圈差异、用户行为分布、季节性），模型训练效果可迁移
- 异动检测验证通过人工注入异常点，可控性更强

兜底策略：代码中所有数据读取走 `DataService` 抽象类，真实对接时只改注入实现，不改 Agent 逻辑。

ADR-003：商品评分模型选 XGBoost 而非深度学习

决策时间：2026-05-07

原因：

- 特征维度低（5维评分），不需要深度模型的表达能力
- XGBoost 输出特征重要性，满足“为什么推荐这个商品”的可解释性需求
- 训练快（秒级），迭代快，适合 MVP 快速试错
- 冷启动场景可以和规则引擎混合（规则兜底），XGBoost 天然支持这种混合架构

已知限制：真实数据量增大后，深度学习可能在排序效果上超越 XGBoost，但 MVP 阶段不需要。

ADR-004：前端选 Streamlit 而非 React/Vue

决策时间：2026-05-07

原因：

- MVP 面向内部运营团队，不需要消费级 UI 精细度
- Python 全栈，Agent 开发者可以直接写前端，不需要前后端分离的人力成本
- Streamlit 的数据图表生态（Plotly 集成）和数据看板模式天然契合
- `st.session_state` 可以管理多页面状态，不需要引入 Redux/Zustand

已知限制：Streamlit 每次交互重新执行脚本，在大量数据渲染时性能差。后续漏斗看板如果数据量大（>10万行），考虑加 `st.cache_data` 或切换到 Plotly Dash。

ADR-005：引入 Planner Agent（编排器Agent）+ 标准化任务协议

决策时间：2026-05-07

来源：GPT-5.5 方案碰撞后吸收

背景：原方案中 workflow 是硬编码的线性流程（选品→定价→文案→审核）。但不同活动目标（拉新 vs 清库存 vs 提ROI）需要不同的 Agent 组合和顺序。

决定增加 Planner Agent：

- 运营只需描述活动目标（目标类型、商圈、预算、品类范围、约束条件）
- Planner Agent 将活动目标拆解为标准子任务清单，决定调用哪些 Agent、传什么参数
- Planner 输出任务清单 + 人审卡点位置 + 每个 Agent 的输入参数
- 这样不同活动类型可以走不同的 workflow 路径，不需要改代码

标准化任务协议：每个 Agent 间的通信使用统一的 JSON 协议：

```
{
  "task_id": "task_001",
  "workflow_id": "wf_20250101_001",
  "agent_name": "selection_agent",
  "biz_goal": "提升ROI",
  "input": { ... },
  "constraints": { "gross_margin_floor": 0.18, "inventory_min": 50 },
  "output_schema": [ "sku_id", "recommend_score", "expected_gmv", "risk_level", "reason" ],
  "confidence_threshold": 0.7,
  "human_review_required": true
}
```

为什么重要：没有协议标准化，每个 Agent 的输入输出格式各异，后期接新 Agent 或修改 workflow 成本极高。协议是 Multi-Agent 系统的"API 契约"。

ADR-006：采用三阶段分步策略（MVP→增强→闭环），不一步到位

决策时间：2026-05-07

来源：GPT-5.5 方案碰撞后吸收

背景：原方案是一次性 6 周交付相对完整的 MVP（含推荐模型、AB实验、投放策略）。GPT-5.5 的方案更务实——先跑通流程，再补智能。

调整后的分阶段策略：

阶段	目标	核心交付	工期
MVP	跑通主链路，辅助决策+人审	活动 workflow + 选品规则引擎 + 文案Agent + 审核Agent + 漏斗看板V1 + 告警中心V1	3-4周
增强	补齐模型能力，建议更准	选品推荐模型(LightGBM) + 定价Agent + 诊断Agent + RAG知识库 + 人工审核台	3-4周
闭环	形成"执行→监控→复盘→沉淀"闭环	复盘Agent + Playbook库 + AB实验模块 + Prompt模板管理 + Agent执行回放	2-3周

核心理念：不追求"全自动"，先做"辅助决策+人审卡点"。运营要有最终决策权和修改能力。

ADR-007: Agent统一输出规范 (evidence + confidence + risk_level)

决策时间: 2026-05-07

来源: GPT-5.5 方案碰撞后吸收

背景: 原方案没有统一输出规范，每个 Agent 输出字段各异，审计和复盘困难。

决定: 所有 Agent 输出必须包含以下 6 个字段：

- `result`：Agent 的核心输出（选品清单、定价建议、文案等）
- `evidence`：决策依据（引用了哪些数据、基于什么规则）
- `confidence`：置信度 0-1（XGBoost概率输出 / LLM自评）
- `risk_level`：风险评级（low/medium/high）
- `recommended_action`：给运营的建议动作
- `need_human_review`：是否需要人工审核

为什么重要：统一规范后才能做审计日志回放、才能做模型评估、才能让运营信任系统。如果每个 Agent 输出格式不同，后期复盘和优化几乎不可行。

二、关键设计模式

模式1: Agent 基类抽象

```
class BaseAgent:
    """所有 Agent 的基类，统一 LLM 调用、工具绑定、日志"""
    def __init__(self, llm, tools: List[Callable], system_prompt: str)
    def invoke(self, state: dict) -> AgentOutput # 返回统一的AgentOutput
    def stream(self, state: dict) -> Generator
```

为什么重要：7个 Agent（Planner/选品/定价/文案/审核/诊断/复盘）如果各自写 LLM 调用逻辑，后期改 prompt 模板、换模型、加日志时会是一场灾难。统一基类后改一处生效全部。

模式2: 人机卡点模式

```
# LangGraph 中的标准模式
def review_node(state: CampaignState) -> CampaignState:
    # Agent 完成审核建议
    state["review_suggestion"] = review_agent.invoke(state)
    return state

# 在图定义中
workflow.add_node("selection_review", lambda s: s) # 卡点节点
workflow.add_edge("selection_agent", "selection_review")
# 关键: 在 review 节点后设置 interrupt
workflow.compile(checkpointer=checkpointer, interrupt_before=["selection_review", "final_review"])
```

运营在前端审批后，调用 `graph.invoke(Command(resume={"approved": True}), config)` 继续。

模式3：模型 + 规则双引擎

选品推荐 = XGBoost 打分 + 规则过滤（库存>0、合规检查、价格带校验）。
模型提供排序，规则提供安全边界。两者解耦，规则可以独立更新。

核心原则（来自 GPT-5.5）：结构化决策靠规则/模型，LLM 负责理解、编排、解释和生成。不要让 LLM 做数学计算或精确排序。

模式4：标准化任务协议模式

所有 Agent 间通信走统一 TaskProtocol，不再各自定义格式：

- Planner Agent 生成任务协议 JSON → 传给下游 Agent
- 每个 Agent 消费协议中的 `input` 和 `constraints`，按 `output_schema` 输出结果
- 下游 Agent 可以从上游 Agent 的 `result` 中读取上下文
- 人审卡点时，前端按协议中的 `output_schema` 渲染审阅界面

模式5：漏斗分层归因（诊断Agent专用）

诊断 Agent 不应笼统地"找原因"，而应按漏斗层级拆解：

异常层级	检测指标	归因方向
曝光下滑	曝光UV	预算不足/渠道分发异常/排位下降/大盘流量波动
CTR下滑	点击UV/曝光UV	文案吸引力不足/图片素材问题/价格不具竞争力/人群错配
加购率下滑	加购UV/点击UV	商品卖点弱/评价差/价格敏感/竞品打折
支付转化下滑	支付UV/加购UV	库存不足/优惠券异常/结算链路问题/配送时效问题

先通过规则+时序模型定位到具体漏斗层级，再由 LLM 做归因解释和建议生成。不要纯靠 LLM "猜测"。

模式6：ChatOpenAI 的 LLM Provider 切换模式

当需要将 LLM 从 OpenAI 切换到 DeepSeek 等其他兼容 OpenAI API 的服务时，无需改 LangChain 调用代码，只需通过 `base_url` 参数：

```
llm_kwargs = {"api_key": settings.LLM_API_KEY, "model": settings.LLM_MODEL, "temperature": 0.3}
if settings.LLM_PROVIDER != "openai":
    llm_kwargs["base_url"] = settings.LLM_API_BASE
self.llm = ChatOpenAI(**llm_kwargs)
```

配置通过环境变量注入，不污染 Agent 逻辑。关键：不是所有 provider 都支持所有 OpenAI 参数（如 `response_format`），切换后需验证结构化输出能力。

模式7：人工审核部分通过模式（Partial Approval）

运营在选品审核时可以逐商品打勾/取消，只通过部分商品。实现要点：

1. **前端**：checkbox 绑定 `session_state` 字典，审批时同步 widget 值 → 读取 → 传 `approved_skus/rejected_skus` 给后端
 2. **后端**：审批端点接收到 `approved_skus` 后，用 `update_state` 过滤 `selection_output.result.selection_list` 为仅已通过商品，再 resume 到下游 Agent
 3. **下游 Agent**：pricing/copywriting/review 直接从 state 中读取 `selection_list`，自动只处理已通过商品，无需改动
- 关键设计：过滤发生在审批端点（而非下游 Agent 内部），下游 Agent 完全无感——它们只看到最终通过的清单。

模式8：ML 模型与规则引擎双模切换

当引入 ML 模型（如 LightGBM）替代规则评分时，不应该直接删除规则引擎——应设计为双模切换：

```
# 启动时选择模型
_lgbm_model = LightGBMScoringModel()
if _lgbm_model.is_trained:
    SCORING_MODEL = _lgbm_model
else:
    SCORING_MODEL = ProductScoringModel() # 规则引擎 fallback
```

ML 模型的 `get_top_n` 方法内部也包含 try/except fallback：

```
def get_top_n(self, ...):
    try:
        df = self.score(...) # LightGBM推理
    except RuntimeError:
        return ProductScoringModel().get_top_n(...) # 降级
```

这样 ML 模型训练失败或文件缺失不会阻塞 workflow，系统始终可运行。

模式9：RAG 知识注入 Agent Prompt

RAG 不是独立产品，而是 Agent 的上下文增强层。注入点放在 Agent invoke 的最外层：

```
def invoke(self, task):
    # 1. 检索相关知识
    rag_context = kb.retrieve_for_context(
        campaign_goal=task.biz_goal,
        district_type=district_name,
        category_scope=category_scope,
    )
    # 2. 注入 system prompt 末尾
    self.system_prompt = BASE_PROMPT + district_context + rag_context
    # 3. 正常 LLM 调用
    return super().invoke(enriched_task)
```

知识注入对 Agent 核心逻辑完全透明——Agent 不知道也不需要知道知识来自 RAG 还是硬编码。检索策略：先精确匹配（目标+商圈+品类），无结果时扩大范围搜索。

模式1：Agent 基类抽象

```
class BaseAgent:
    """所有 Agent 的基类，统一 LLM 调用、工具绑定、日志"""
    def __init__(self, llm, tools: List[Callable], system_prompt: str)
    def invoke(self, state: dict) -> dict
    def stream(self, state: dict) -> Generator
```

为什么重要：5个 Agent 如果各自写 LLM 调用逻辑，后期改 prompt 模板、换模型、加日志时会是一场灾难。统一基类后改一处生效全部。

模式2：人机卡点模式

```
# LangGraph 中的标准模式
def review_node(state: CampaignState) -> CampaignState:
    # Agent 完成审核建议
    state["review_suggestion"] = review_agent.invoke(state)
    return state

# 在图定义中
workflow.add_node("review", review_node)
workflow.add_edge("copywriting", "review")
# 关键：在 review 节点后设置 interrupt
workflow.compile(checkpointer=checkpointer, interrupt_before=["review"])
```

运营在前端审批后，调用 `graph.invoke(Command(resume={"approved": True}), config)` 继续。

模式3：模型 + 规则双引擎

选品推荐 = XGBoost 打分 + 规则过滤（库存>0、合规检查、价格带校验）。
模型提供排序，规则提供安全边界。两者解耦，规则可以独立更新。

三、已知陷阱与注意事项

陷阱1：LLM 输出格式不稳定

表现：要求 Agent 输出结构化 JSON，但 LLM 偶尔输出多余文字或格式错误。

对策：

- 使用 LangChain 的 `with_structured_output()` 或 OpenAI 的 `response_format={"type": "json_object"}`
- 在 Agent 基类中统一加后处理：解析 JSON 失败时重试一次，再失败走规则兜底
- 关键字段（价格、折扣率）用规则二次校验，不信任 LLM 的输出

陷阱2：Celery + LangGraph 的状态一致性

表现：定时诊断任务（Celery Beat）触发诊断 Agent，Agent 运行时如果服务重启，checkpoint 可能不一致。

对策：

- 诊断 Agent 设计为无状态（每次运行独立，不依赖上次诊断结果）
- 如果需要跨诊断保留上下文，用 SQLite 而非 LangGraph checkpointer

陷阱3：模拟数据过于"干净"

表现：模拟数据没有真实数据中的噪音和异常值，训练出的模型在真实场景表现差。

对策:

- 模拟数据生成器中加入一定比例的噪音 (5%缺失值、2%异常值)
- 特征工程中显式处理缺失值和异常值 (不依赖数据完美)
- 在 Phase 3 的 AB 实验中留出"人工组", 对比效果时保持谦逊

陷阱4: Streamlit session_state 的多页面状态污染

表现: Streamlit 的多页面应用共享 `st.session_state`, 不同页面的 key 可能冲突。

对策:

- 使用前缀命名空间: `st.session_state["workflow_campaign_id"]`
- 在页面切换时显式清理不相关的 session state

陷阱5: 一上来就追求"全自动" (来自 GPT-5.5 警告)

表现: 试图让 Agent 直接决策并执行 (自动选品上架、自动定价生效), 运营失去控制权, 系统一出错就造成实际损失。

对策:

- MVP 阶段所有决策必须经过人审卡点, 模型只做"推荐"不做"决策"
- 文案生成、定价建议都标注置信度和风险评级, 低置信度强制人工审核
- 逐步放开自动化: 先"辅助建议"→再"机器初审+人工复核"→最后"高置信自动执行+异常回滚"
- 定价类敏感操作必须有硬约束 (毛利红线、爆款改价二次确认)

陷阱6: 没有反馈闭环, 模型无法持续变好 (来自 GPT-5.5 警告)

表现: 运营用了推荐但不反馈好坏, Agent 不知道自己做得对不对, 模型效果停滞甚至退化。

对策:

- 每个 Agent 输出后必须有"采纳/拒绝"按钮, 拒绝时必须填写原因
- 记录 agent_feedback 表 (run_id, agent_name, accepted, reason, result_quality)
- 定期分析采纳率和拒绝原因, 驱动 prompt 优化和模型重训
- 复盘 Agent 自动将成功活动经验写入 Playbook 库

陷阱7: 指标口径不统一导致 Agent 失效

表现: 同一个"转化率", 运营说的是支付/曝光, 算法说的是支付/加购, Agent 在不同上下文拿到不同口径的数据, 归因完全错误。

对策:

- 在项目中建 metrics_definition.md, 统一定义所有指标口径
- 数据服务层 (MetricsService) 是唯一的指标计算入口, 禁止各 Agent 自行计算
- 输出中附上指标口径标识 (如 `cvr_pay_imp` vs `cvr_pay_cart`)

陷阱8: LangGraph SqliteSaver v2.0+ API 变更

表现: `SqliteSaver.from_conn_string(db_path)` 报 `AttributeError: '_GeneratorContextManager' object has no attribute 'get_next_version'`。

对策: langgraph-checkpoint-sqlite 2.0+ 的 SqliteSaver 不再接受路径字符串, 必须传入 `sqlite3.Connection` 对象:


```
import sqlite3
conn = sqlite3.connect(db_path, check_same_thread=False)
checkpointeer = SqliteSaver(conn)
```

陷阱9：LangGraph interrupt 后必须用 Command(resume=...) 而非 None

表现：`campaign_graph.invoke(None, config)` 在 interrupt 后报错，无法继续执行。

对策：LangGraph 0.2+ 在 interrupt 后的恢复调用必须使用 Command：

```
from langgraph.types import Command
result = campaign_graph.invoke(Command(resume={"selection_approved": True}), config)
```

不能传 `None` 或空字典。`Command(resume=...)` 是唯一的恢复方式。

陷阱10：pandas Series 的 truth value 在迭代比较中报错

表现：`month in [6, 7, 8]` 当 `month` 是 pandas Series 时，pandas 无法判断 Series 的布尔值，报 `ValueError: The truth value of a Series is ambiguous`。

对策：涉及 pandas 数据的条件判断不要用 Python 原生 `in`，改用向量化的 `.isin()`：

```
# 错误：iterrows 中逐行判断
for _, row in df.iterrows():
    if row["month"] in [6, 7, 8]: # 如果 month 列是 Series 会报错

# 正确：向量化操作
summer_mask = df["category"].isin(["饮料", "水果", "酒类"])
season_score[summer_mask] = 0.8
```

陷阱11：Streamlit checkbox 的 on_change lambda 捕获过期值

表现：在循环中用 lambda 做 checkbox 的 `on_change` 回调时，lambda 捕获的变量值（如 `sku`、`cid`）在回调触发时已变为循环最后一次的值，导致所有 checkbox 都操作同一个元素。

对策：不要在 Streamlit checkbox 上使用 `on_change` 做列表同步。改为在点击审批按钮时统一步 checkbox 值到 `session_state` 字典：

```
# 渲染时不加 on_change
cols[0].checkbox("✓", value=..., key=f"chk_{cid}_{sku}", label_visibility="collapsed")

# 点击审批按钮时统一步
for p in selection_list:
    sku = p["sku_id"]
    st.session_state[f"sel_checkboxes_{cid}"][sku] = st.session_state.get(f"chk_{cid}_{sku}", True)
```

默认的变量默认参数技巧（`lambda s=sku: ...`）在 Streamlit 回调中不可靠，因为 widget 状态更新和回调执行时机不确定。

陷阱12：前端函数签名与后端 API 参数不一致导致静默失败

表现：前端 `approve_selection(cid, True, approved_skus, rejected_skus)` 调用传了 4 个参数，但函数签名仍是 `def approve_selection(campaign_id, approved, comment="")`——多余的参数被 httpx 忽略，后端收不到 `approved_skus/rejected_skus`，但不会报错，调试极难。

对策：任何前后端交互的字段变更必须三步同步：

1. 前端 helper 函数签名 → 2. 前端 httpx 请求 body → 3. 后端 Pydantic 模型。改一必改三，漏一即静默失败。建议在函数签名中显式声明所有参数而非用 `**kwargs`，让 IDE 和 linter 能检测不一致。

四、开发流程约定

开发前 checklist

- ☐ 阅读本文，确认是否已有相关经验
- ☐ 确认当前 Phase 目标，不越界开发
- ☐ 检查是否有需要先完成的前置任务

开发后 checklist

- ☐ 更新本文：新增 ADR / 陷阱 / 模式
- ☐ 代码中关键决策加注引用本文的 ADR 编号
- ☐ 如果有推翻之前 ADR 的决策，明确标注"已废弃"及原因

代码规范

- Agent 的 Prompt 模板统一放在 `backend/app/agents/prompts/` 目录下，不硬编码
- 特征工程相关代码必须实现 `FeatureExtractor` 基类，方便后续替换数据源
- 所有 LLM 调用的 Token 用量记录到日志，方便成本核算

五、当前进度

阶段	状态	目标	完成日期
Phase 0：基础设施 + 数据底座	✅ 完成	项目脚手架 + 模拟数据 + 漏斗看板V1	2026-05-07
Phase 1：MVP 核心 workflow	✅ 完成	Planner+选品+文案+审核 workflow + 人审卡点	2026-05-07
Phase 2：智能增强	✅ 完成	诊断Agent + LightGBM + RAG知识库	2026-05-08

Phase 1 经验沉淀（2026-05-07 ~ 2026-05-08）

已完成：

- 特征引擎（5类特征提取：商圈/商品/需求/竞品/场景），`FeatureExtractor` 基类抽象
- 商品评分模型（5维加权：销售潜力0.30+毛利空间0.25+竞争稀缺性0.20+季节匹配0.15+库存健康0.10）
- Planner Agent（活动目标→任务DAG+人审卡点）
- Selection Agent（模型打分+LLM推理，双层：规则层推荐Top30→LLM精选+解释）
- Pricing Agent（双层：规则层硬约束+LLM定价建议）
- Copywriting Agent（商圈人群适配+多版本输出）
- Review Agent（规则检查+LLM综合审核）
- LangGraph 工作流（7节点：planner→selection→checkpoint→pricing→copywriting→review→final_checkpoint，2个人审卡点）
- 工作流 API（创建/状态查询/选品审批/终审批复）

- Streamlit workflow 操作台（创建表单→进度条→选品审核面板→终审面板→驳回重做）
- DeepSeek API 集成（通过 ChatOpenAI 的 base_url 参数，无需改 LangChain 调用代码）
- 选品逐商品打勾审批（checkbox + 全选/取消全选 + 部分通过/部分驳回）

关键决策：

- interrupt_before 放在 checkpoint 节点而非 agent 节点（agent 执行后才暂停，运营看到的是结果而非空白状态）
- Selection Agent 先跑模型打分再把结果喂给 LLM，实现了"规则+LLM 双层"模式（ADR-003）
- Pricing Agent 的 _apply_pricing_rules 是纯规则函数，不依赖 LLM，保证了毛利下限等硬约束
- 条件边 route_selection 和 route_final 实现了"驳回→重做"循环
- 部分审批模式：通过后自动过滤 selection_output 为仅已通过 SKU，下游 Agent 只处理已通过商品

已知问题：

- LLM 依赖 API_KEY，未设置时 Agent 会失败。Phase 2 考虑加本地模型 fallback
- SqliteSaver 在多并发时可能有锁冲突，MVP 单用户使用无影响
- Streamlit workflow 页面刷新机制靠 st.rerun()，长流程可能有卡顿感

Phase 2 经验沉淀（2026-05-08）

已完成：

- 异动检测引擎（AnomalyDetector，3种检测方法：Z-score 统计带、绝对值底线、日环比降幅，13个指标，4级严重程度）
- 漏斗归因引擎（FunnelAttributor，16条归因规则矩阵，按漏斗层匹配异常到根因类别）
- 诊断 Agent（DiagnosisAgent，规则引擎 + LLM 双层：检测→归因→LLM分析建议）
- 诊断 API（/run、/anomalies、/reports、/reports/{id}）
- 诊断看板（异常卡片 + 漏斗归因展开 + LLM建议 + 历史报告，完整深度）
- 模拟数据异动注入（inject_diagnosis_test_anomalies，5个商圈各注入已知异动模式）
- LightGBM 排序模型（LightGBMScoringModel，pickle持久化，特征重要性输出，规则引擎 fallback）
- LightGBM 训练流水线（MockFeatureExtractor 提取~20维特征 → GMV/CVR/Margin复合目标 → LightGBM回归训练）
- SelectionAgent 集成 LightGBM（自动检测模型是否训练，未训练时 fallback 规则引擎）
- RAG 知识库（KnowledgeBase，ChromaDB + 内存fallback，6篇种子文档：策略playbook + 异常修复案例）
- RAG API（/api/rag/search、/ingest、/stats、/seed）
- SelectionAgent 注入 RAG 上下文（根据活动目标和商圈检索历史 playbook 经验）

关键决策：

- LightGBM 与规则引擎并存：模型未训练时自动 fallback 规则引擎，不阻塞现有 workflow
- RAG 用 ChromaDB + 内存 fallback 双模式，即使 ChromaDB 未安装也能运行
- 诊断 Agent 的规则引擎层（异动检测+漏斗归因）不依赖 LLM，可独立工作；LLM 层仅在「运行诊断」时启用
- 种子文档覆盖 3 种类型（playbook/strategy/anomaly_fix），为后续复盘 Agent 打基础

新增陷阱：

- 陷阱13：pandas groupby 后的索引访问需注意 MultiIndex 情况
- 陷阱14：LightGBM pickle 保存需确保 sklearn/lightgbm 版本兼容，跨版本可能加载失败
- 陷阱15：ChromaDB 的 embedding function 默认使用 all-MiniLM-L6-v2，需联网下载模型；离线环境需预下载或切换 embedding

| Phase 3：闭环运营 |  待开始 | 复盘Agent+Playbook+AB实验+Agent回放 | - |

Phase 0 经验沉淀（2026-05-07）

已完成：

- FastAPI 项目骨架 + LangGraph Agent 基类（BaseAgent / AgentOutput / TaskProtocol）
- SQLite 数据库设计（11张表，含 agent_feedback、agent_run_log 治理表）
- 模拟数据生成器（5商圈×200商品×10000用户×3月漏斗数据，含 5% 噪音 + 2% 异常注入）
- 指标口径文档（12 北极星 + 6 护栏 + 漏斗定义 + 时间粒度）
- MetricsService 作为唯一指标计算入口，禁止 Agent 自行计算
- DataService 抽象层，MockDataService 实现
- Streamlit 漏斗看板 V1（指标卡 + 漏斗图 + GMV趋势 + 漏斗各层趋势）

关键决策：

- interrupt_before 应该在 workflow 定义阶段就规划好人审卡点位置，而不是事后加
- BaseAgent._log_run 记录到 agent_run_log 表，为 Phase 3 的 Agent 回放打基础
- 模拟数据生成时 district_cvr 按商圈差异化（国贸高、五道口低），让数据看起来真实

六、外部参考资料

GPT-5.5 方案碰撞要点（2026-05-07）

与 GPT-5.5 的方案对比后，以下关键点被吸收进本方案：

1. **Planner Agent + 标准化任务协议**：这是原方案最大的遗漏。没有编排器，工作流就是硬编码的，不同活动类型无法灵活适配。
2. **统一输出规范（evidence/confidence/risk_level）**：原方案各 Agent 输出自由度过高，不利于审计和迭代。
3. **三阶段分步策略**：原方案追求一次性交付较完整的 MVP，调整为"先跑通流程→再补智能→最后闭环"。
4. **漏斗分层归因**：诊断 Agent 不能笼统归因，必须定位到具体漏斗层级。
5. **反馈闭环设计**：agent_feedback 表和采纳率追踪是模型持续优化的基础。
6. **治理层**：审计日志、Prompt版本管理、Agent执行回放——这些在原方案中被忽略，但对产品化至关重要。
7. **定价Agent的多层约束设计**：毛利红线、爆款改价二次确认、高波动价格审批——这些业务卡点不是技术细节，是产品安全边界。
8. **复盘Agent + Playbook库**：原方案只提到"沉淀运营Playbook"，但没设计具体的复盘Agent和知识库结构。

尚未吸收但值得后续考虑的：

- 真实业务 Shadowing（运营流程观察）——这是 Phase 0 应该做的，但纯开发场景可能跳过
- 数据仓库分层（ODS/DWD/DWS/ADS）——MVP 数据量小暂不需要，生产环境必须
- 消息队列（Kafka）——MVP 用 Celery+Redis 够用，生产环境考虑
- 对象存储和监控——MVP 暂不需要

最后更新：2026-05-08

本次更新：Phase 2 完成——诊断Agent + LightGBM模型 + RAG知识库 + 陷阱13-15 + 模式8-9

下一篇文章预期：Phase 3 开发——复盘Agent、Playbook库、AB实验模块、Agent执行回放